Chapter 10: Object-oriented programming

Khiem Nguyen – khiem.nguyen@glasgow.ac.uk

Contents

1 Why Object-Oriented Programming **1**
1.1 Procedure programming vs Object-oriented programming . . . 1
1.2 The dominance of OOP . . . 1
1.3 C++ is highly desired in industry . . . 3
1.4 C++ is highly desired in industry: Software architecture at Airbus . . . 4
1.5 Git Control and OOP are essential skills in industry . . . 4
1.6 Why OOP: 4 key pillars of OOP . . . 5
1.7 Encapsulation . . . 5
1.8 Encapsulation . . . 6
1.9 Encapsulation . . . 6
1.10 Abstraction . . . 7
1.11 Abstraction . . . 8
1.12 Encapsulation vs Abstraction . . . 9
1.13 Inheritance . . . 9
1.14 Polymorphism . . . 9

2 `struct` vs `class` **10**
2.1 `struct` vs `class`: Technical difference (defaults) . . . 10
2.2 `struct` vs `class`: Google C++ style . . . 10
2.3 `struct` vs `class`: Google C++ style . . . 11

3 Defining a class: Syntax **11**
3.1 Defining a class . . . 11
3.2 Naming convention: Google style . . . 11
3.3 Naming convention: Which to choose . . . 12
3.4 Naming convention: Which to choose … and But . . . 12
3.5 A `class` can implement a `struct` . . . 12
3.6 Most of the C++ standard library is classes . . . 12

4 Member functions and implicit objects **13**
4.1 Member functions . . . 13
4.2 A non-member function example . . . 13
4.3 A member function example . . . 13
4.4 Calling member functions and the implicit object . . . 14
4.5 Assessing members inside a member function uses the implicit object . . . 14
4.6 A comparison to OOP in Python . . . 16
4.7 A comparison to OOP in Python: Repeat last slide . . . 16
4.8 A comparison to OOP in Python: But Python is not as serious as C++ . . . 16
4.9 A comparison to OOP in Python: But Python is not as serious as C++ . . . 17
4.10 Member functions can be overloaded . . . 17
4.11 Structs and member functions: A bit of history . . . 17

5 Const class objects and const member functions **17**
5.1 Const variables: A quick revisit . . . 18
5.2 Modifying the data members of const objects is disallowed . . . 19
5.3 Const objects may not call non-const member functions . . . 19
5.4 Const member functions . . . 20
5.5 Const member functions . . . 20
5.6 Const member functions can be called on non-const objects . . . 20
5.7 Const member functions can be called on non-const objects . . . 20
5.8 Const member functions can be called on non-const objects . . . 20
5.9 Wait a minute: What a mess . . . 21
5.10 Const objects via pass by const reference . . . 21
5.11 Const objects via pass by const reference . . . 22
5.12 Const objects via pass by const reference . . . 22

1 Why Object-Oriented Programming

1.1 Procedure programming vs Object-oriented programming

- Up to now, we've been doing a type of programming called **procedure programming**. In procedure programming, the functions and the data those functions operate on are separate entities.
- In **object-oriented programming** (OOP), the focus is on creating program-defined data types that contain both properties and a set of well-defined behaviors.

We won't discuss why we need Object-Oriented Programming (OOP) in this course:

- We have all learned the importance of OOP in "Introductory Programming 2" and other programming courses.
- Since the 1990s, OOP has been the industry standard.

Instead, I walk you through some job posts in software engineer industry.

1.2 The dominance of OOP

- **Why it's popular**: It mimics how we perceive the world – objects with properties and behaviors.
- **Scalable**: OOP makes it possible to manage large, complex systems where we need to "encapsulate" data to prevent bugs.
- **The industry "lungs"**: If you look at job postings or legacy codebases, OOP is everywhere.

Responsibilities

After training on MathWorks development processes and tools, you will work with a development team on implementing features such as algorithms or UIs for a future release of one of our 100+ products, or help to develop automated tests to verify that features in MATLAB or Simulink are working as designed.

This internship will prepare you for a career at MathWorks starting in our Engineering Development Group (EDG).

Qualifications

- Pursuing a MS or PhD in Computer Science, Engineering, or another STEM subject, and be enrolled in education at the time of the internship
- Experience with object orientated design and in programming languages such as: C++, JavaScript, Python, Go or MATLAB
- Passion for learning new technologies

You don't need to know MATLAB to find a job in Mathworks. Instead, you need experience in C++ and OOP.

1.3 C++ is highly desired in industry

I asked Gemini about software engineering roles at MathWorks (developing MATLAB you use all the time.)

It is **very right** to say that for most core software engineering roles at MathWorks, C++ is a primary requirement.

Because MathWorks builds high-performance tools like **MATLAB** and **Simulink**, they need a language that can handle heavy numerical computation and memory management—which is exactly what C++ is designed for.

Here is a breakdown of why C++ is so critical there and what else they look for:

---

1. Why MathWorks depends on C++

Most of the "under the hood" engine of MATLAB and Simulink is written in **C++** and **C**.

- **Performance:** High-speed matrix math and simulation require the low-level control that C++ provides.
- **Code Generation:** Many MathWorks products (like MATLAB Coder) are specifically designed to turn MATLAB code *into* C++ code for use in cars, planes, and medical devices.
- **Legacy & Scale:** With millions of lines of code, their infrastructure relies on the robust Object-Oriented patterns found in modern C++.

---

2. When is C++ *not* required?

While C++ is the "king" at MathWorks, it isn't the only way in. Depending on the team, they look

1.4 C++ is highly desired in industry: Software architecture at Airbus

ABOUT YOU

- Bachelor's or Master's degree in Computer Science, Software Engineering, or a related technical field.
- 8-10 years of progressive experience in software development, with at least 3-5 years in a dedicated Software Architect role.
- Proven experience in designing and delivering complex, large-scale software systems, preferably within the defence, aerospace, or other highly regulated industries.
- Demonstrable experience with various architectural patterns (e.g., microservices, event-driven, monolithic decomposition) and their appropriate application.
- Strong proficiency in multiple programming languages (e.g., C++, Java, Python, Ada) relevant to embedded or real-time systems.
- Extensive knowledge of software design principles, object-oriented design, and design patterns.
- Experience with real-time operating systems (RTOS) and embedded software development is highly desirable.
- Familiarity with cloud computing platforms (e.g., AWS, Azure, Google Cloud) and containerization technologies (e.g., Docker, Kubernetes) where applicable.
- Understanding of cybersecurity principles and secure software development practices.
- Experience with model-driven architecture (MDA) tools and techniques is a plus.

Not a 100% match? No worries! Airbus supports your personal growth with customised development solutions.

1.5 Git Control and OOP are essential skills in industry

Note the requirement of experience with Java. For your record, Java is a pure OOP programming language.

ABOUT YOU

- Bachelor's or Master's degree in Computer Science, Software Engineering, or a related technical field.
- Proven experience in software development, preferably within a professional environment.
- Strong experience with Java (17+),
- Experience with software development methodologies (e.g., Agile, Scrum).
- Familiarity with version control systems (e.g., Git).
- Understanding of software testing principles and experience with unit and integration testing.

Not a 100% match? No worries! Airbus supports your personal growth with customised development solutions.

1.6 Why OOP: 4 key pillars of OOP

I hope this is more than enough to convince you why we need to learn Object-Oriented Programming, especially in C++.

But that's a very lazy explanation.

There are four **key pillars in Object-Oriented Programming**

1. **Encapsulation**
2. **Abstraction**
3. **Inheritance**
4. **Polymorphism**

We shall quickly go through each of them, explain what they are and why they are.

1.7 Encapsulation

Encapsulation is the practice of bundling data (variables) and methods (functions) that operate on that data into a single unit called **class**.

It involves "hiding" the internal state of the object from the outside world to prevent accidental interference.

- **Example 1** Remember what you learned in Python:
  - `ndarray` object in NumPy library
  - `Figure` object is returned when you plot `pyplot.plot(x, y)`
- **Example 2** Think of this code snippet `a = np.array([1, 'two', 3])`:
  - The interpreter tells you the error and *exactly what kind of error you made.*
  - This is because there is *a ton of checking behind the scene*, and *the user does not need to know.*
- **Example 3** Think of registering a student at UoG (python): `Student(name="Bull Shit", age=7)`
  - There must be some behind-the-scene checking, or otherwise mistakes can easily slip through.
  - Inform the user that something is wrong.

1.8 Encapsulation

In C++, we use **access specifiers** to achieve this:

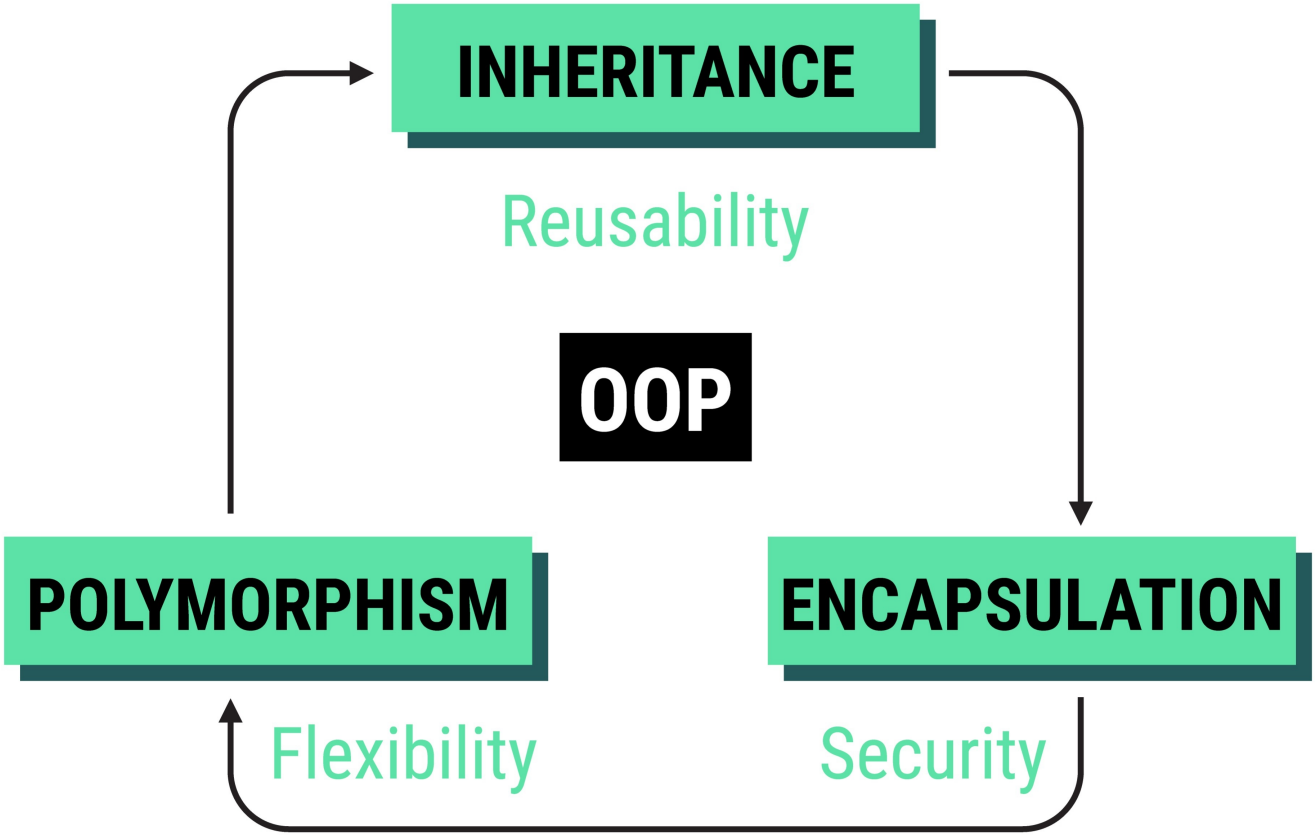
`private`: Members are only accessible within the class.

`public`: Members are accessible from outside the class.

- **Example** Think of a **Bank Account**
  - You should not be able to change the `balance` variable directly from outside – otherwise, the bank is F____ed up.
  - You should have to use `deposit()` or `withdraw()` method that checks if the transaction is valid.

1.9 Encapsulation

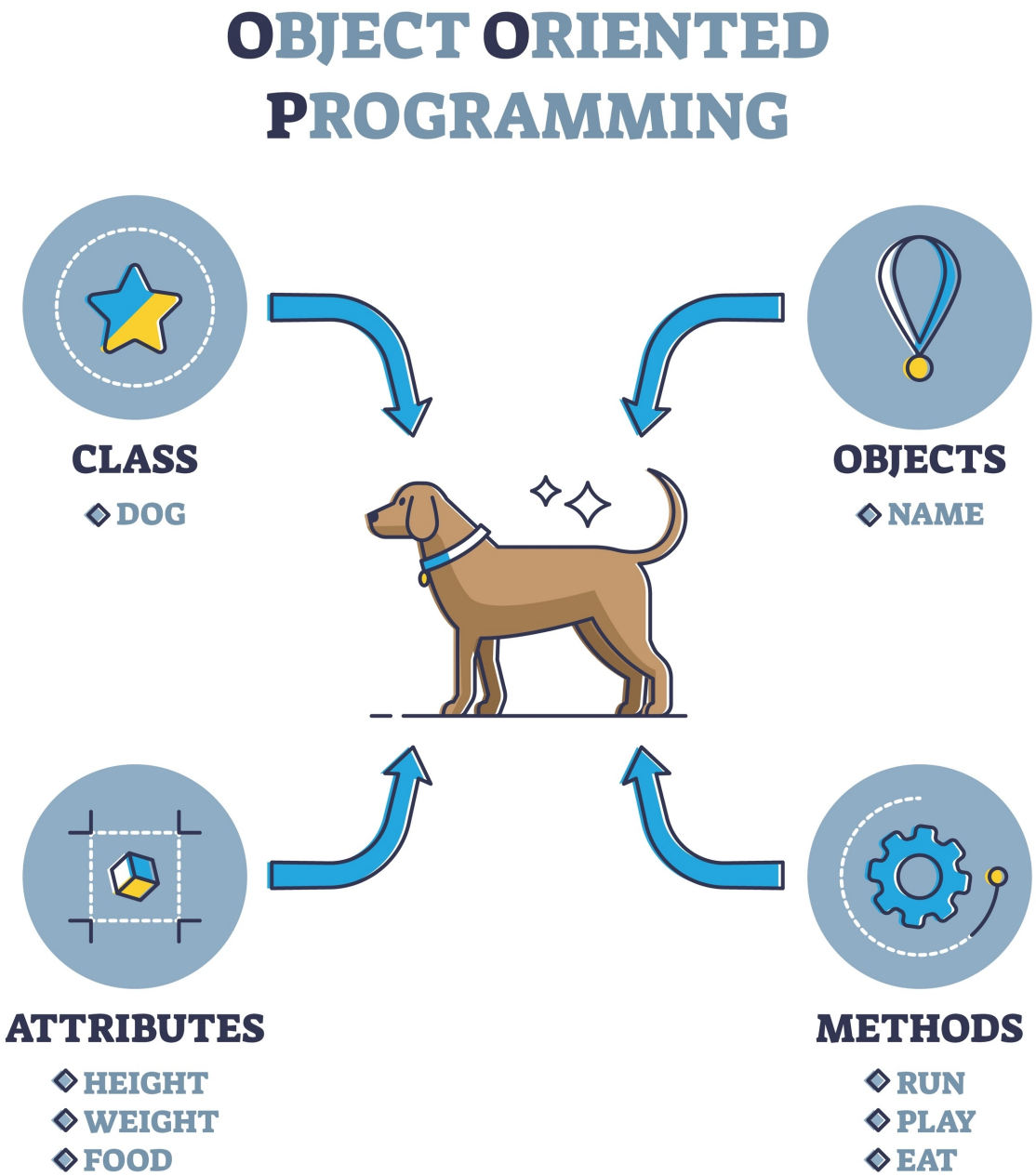
Encapsulation enhances **security**.



1.10 Abstraction

Abstraction is about **simplifying** things.

It means showing only the essential features of an object and hiding the complex background details.



1.11 Abstraction

Why use abstraction

- **Reduces complexity**: We only deal with a high-level interface (from a user point of view)
- **Easy Maintaince**: We can change the internal "engine" of our code (from a developer point of view) without changing how other people use it.

- **Example** Think of a **Coffee Machine**:
  - only need to know which button to press
  - no need to understand the internal water pressure, size of grind bean, etc.

- **Example** We want to build a graphics app:

- We want to "*draw*" different shapes, but the ways we draw a Circle and a Square are different.
- We create an **Abstract Class** called **Shape**. It acts as a blueprint that says: "Anything that is a shape must have a `draw()` method,

```cpp
class Shape {
public:
    // Pure virtual function makes this an Abstract Class
    virtual void draw() = 0;
};
```

1.12 Encapsulation vs Abstraction

- **Encapsulation** is about **protection and data integrity**.
  - **Example** Check if a `battery_level` is negative
  - **Example** Check if a `withdrawl_amount` exeeds a balance
- **Abstraction** is about **interface and simplification**.
  - **Example** Allow user to simply call `set_battery(50)` without having to see the `if (value < 0)` logic hidden inside.

My own opinion:

- I try my best to find examples to illustrate these two points.
- Sometimes, I can get confused by myself.
- Some examples can sit at the intersection of both encapsulation & abstraction.
- Don't worry if you get confused after this

1.13 Inheritance

Inheritance allows us to create a new class (**Derived** class or **Child** class) based on existing class (**Base** class or **Parent** class).

The child class "*inherits*" the attributes and behaviors of the parent, which helps us *avoid writing the same code over and over again* Don't repeat yourself.

- **Example** An circle is an ellipse.
  - A circle is an ellipse with the same length of two axes.
  - If one can draw an ellipse, there is no point to copy/paste the code to draw a circle.
- **Example** Lecturer, Member of Teaching Office, Technician, Demonstrators are all Employee with names, age, salary, etc.
  - combine all common features and behaviors for all kinds of Employee
  - bundle these features and behaviors into Employee
  - no need to repeat code for each kind of Employee

1.14 Polymorphism

Polymorphism allows an object to be treated as an instance of its parent class, but still behave according to its own specific type.

- **Example** We can have a list of different shapes and tell them all to `draw()`, and they will each "know" how to draw themselves correctly.
- We shall understand this pillar better when we reach this point.

2 `struct` vs `class`

Importance Note

In C++, the difference between a `struct` and a `class` is surprising small from a technical standpoint, but quite significant from a **stylistic** and **architecture** standpoint.

- Therefore, any example that is implemented using a struct could be implemented using a class, or vice-versa.
- However, from a practical standpoint, we use `struct` and `class` differently.

2.1 `struct` vs `class`: Technical difference (defaults)

In C++, a `struct` and a `class` are almost identical, except for the default access level of their members and inheritance. That's it!

Default Access Control

- `struct`: Members (variables and functions) are `public` by default
- `class` : Members are `private` by default

Default Inheritance

- `struct`: Inherits `publicly` by default
- `class` : Inherits `privately` by default

We shall study these concepts soon.

2.2 `struct` vs `class`: Google C++ style

Technically, you can use `struct` and `class` interchangably. Practically, we use them with different purposes. Google suggests

Structs vs. Classes

Use a `struct` only for passive objects that carry data; everything else is a `class`.

The `struct` and `class` keywords behave almost identically in C++. We add our own semantic meanings to each keyword, so you should use the appropriate keyword for the data-type you're defining.

`structs` should be used for passive objects that carry data, and may have associated constants. All fields must be public. The struct type itself must not have invariants that imply relationships between different fields, since direct user access to those fields may break those invariants, but users of a struct may have requirements and guarantees on particular uses of it. Constructors, destructors, and helper methods may be present; however, these methods must not require or enforce any invariants.

If more functionality or invariants are required, or struct has wide visibility and expected to evolve, then a `class` is more appropriate. If in doubt, make it a `class`.

For consistency with STL, you can use `struct` instead of `class` for stateless types, such as traits, template metafunctions, and some functors.

Note that member variables in structs and classes have different naming rules.

2.3 `struct` vs `class`: Google C++ style

Key points

- Use a `struct` only for passive objects that carry data; everything else is a `class`.
- The `struct` and `class` keywords behave almost identically in C++. We add our own semantic meanings to each keyword, so you should use the appropriate keyword for the data-type you're defining.

3 Defining a class: Syntax

3.1 Defining a class

- A class is a **program-defined** data type. Sometimes, you hear **user-defined** data type.
- Classes are defined similarly to structs: Classes are defined using the `class` keyword, structs using the `struct` keyword.

```
class Employee
{
    int m_id {};            // m_ stands for member, m_id is the member variable
    int m_age {};
    double m_wage {};
}
```

Naming convention

The prefix `m_` is used to signify the member variables of a class.

We discuss why member variables of a class are often prefixed with an `m_` in upcoming lesson.

3.2 Naming convention: Google style

Class Data Members

Data members of classes, both static and non-static, are named like ordinary nonmember variables, but with a trailing underscore. The exception to this is static constant class members, which should follow the rules for naming constants.

```
class TableInfo {
 public:
  ...
  static const int kTableVersion = 3;  // OK - constant naming.
  ...

 private:
  std::string table_name_;             // OK - underscore at end.
  static Pool<TableInfo>* pool_;       // OK.
};
```

3.3 Naming convention: Which to choose

- Several companies and wide industry have adopted the prefix `m_` naming convention.

```
class Employee
{
    int m_id {};
    int m_age {};
    double m_wage {};
}
```

- Google suggests trailing underscore, such as `id_`, `age_`, and `wage_`

```
class Employee
{
    int id_ {};
    int age_ {};
    double wage_ {}
}
```

When we use the prefix `m_` convention, the intelliSense will suggest variable names when we type `m_`.

3.4 Naming convention: Which to choose … and But

- The prefix `m_` or the suffix `_` help us to avoid **name shadowing** in **class constructors**.
- When you learn more about modern C++ (in our lectures actually), you soon recognize: we don't even need the above naming conventions.

Conclusion:

- I would prefer not to use the prefix naming convention `m_` in our lectures.
- Saying that, sometimes I mix the convention `m_` in on purpose (or maybe by forgetting).

3.5 A `class` can implement a `struct`

By declaring `public` member variables, a `class` behaves just like a `struct`.

3.6 Most of the C++ standard library is classes

We have already been using class objects, perhaps without knowing it:

- `std::string`
- `std::string_view`
- `std::vector`

In fact, most of the non-aliased types in the standard library are defined as classes!

Classes are really the heart and soul of C++ – they are so foundational that C++ was originally named "C with classes"!

Listing 1 A `class` behaves like a `struct` by declaring member variables `public`. Filename=`class_just_like_struct.cpp`

```cpp
#include <iostream>

class Date       // we changed struct to class
{
public:          // and added this line, which is called an access specifier
    int m_day{};        // we can add "m_" prefixes to each of the member
    int m_month{};      // names to avoid shadowing. But this is not really necessary
    int m_year{};       // in modern C++.
};

void print_date(const Date& date)
{
    std::cout << date.m_day << '/' << date.m_month << '/' << date.m_year;
}

int main()
{

    Date date{ 4, 10, 21 };
    print_date(date);

    return 0;
}
```

4 Member functions and implicit objects

- In this Section, we shall use `struct` to demonstrate the concepts.
- By using `struct`, I can demonstrate that `struct` and `class` are technically almost the same.

4.1 Member functions

- In addition to having member variables, `class` types (which include `struct`, `class`, `union`) can have their own functions.
- Functions that belong to a class type are called **member functions**.
- Functions that are not member functions are called **non-member functions**.

4.2 A non-member function example

Let us look at the following example:

4.3 A member function example

In this lesson, we'll use structs to show examples of member functions – but everything we show here applies equally well to classes.

Listing 2 An example of non-member function. Filename=`non_member_function_example.cpp`

```cpp
#include <iostream>

struct Date
{
    // here are our member variables
    int year {};
    int month {};
    int day {};
};

void print(const Date& date)
{
    // member variables accessed using member selection operator (.)
    std::cout << date.year << '/' << date.month << '/' << date.day;
}

int main()
{
    Date today { 2026, 02, 27 }; // aggregate initialize our struct

    today.day = 16; // member variables accessed using member selection operator (.)
    print(today);   // non-member function accessed using normal calling convention

    return 0;
}
```

4.4 Calling member functions and the implicit object

- In the non-member example, we call

```
print(today)
```

- In the member example, we call

```
today.print()
```

- In the member function case, we don't pass `today` as an argument.
- The object is **implicitly** passed to the member function.
- The object that member function is called on is called **implicit object**

4.5 Assessing members inside a member function uses the implicit object

In the non-member example, we define

```cpp
// non-member version of print
void print(const Date& date)
{
```

Listing 3 An example of member function. Filename=`member_function_example.cpp`

```cpp
// Member function version
#include <iostream>

struct Date
{
    int year {};        // We don't use m_ here for struct. Generally, we don't
    int month {};       // want to write constructor for struct. Then, there is
    int day {};         // NO SHADOWING of variables (we talk later in `class`)

    void print() // defines a member function named print
    {
        std::cout << year << '/' << month << '/' << day;
    }
};

int main()
{
    Date today { 2026, 02, 26 }; // aggregate initialize our struct
    today.day = 16; // member variables accessed using member selection operator (.)
    today.print();  // member functions also accessed using member selection operator (.)
    std::cout << '\n';
    return 0;
}
```

```cpp
    // member variables accessed using member selection operator (.)
    std::cout << date.year << '/' << date.month << '/' << date.day;
}
```

In the member example, we define

```cpp
void print() // defines a member function named print()
{
    std::cout << year << '/' << month << '/' << day;
}
```

- We access the members as `year`, `month`, and `day`.
- Any member identifier that is not prefixed with the member selection operator (`.`) is associated with the implicit object.
- `year`, `month`, and `day` (which are not prefixed) evaluate to the values of `today.year`, `today.month`, and `today.day` respectively.

4.6 A comparison to OOP in Python

```python
class Date:
  def __init__(self, year=2026, month=2, date=28): # we learn constructor in C++ soon
    self.year = year
    self.month = month
    self.date = date
  def print(self):
    print(f"{self.year}/{self.month}/{self.date}")  # we don't have self. in C++

# Use this class in global scope
today = Date(2026,2,26) # when this lecture first took place
today.print()
# How about this: I know that many of you did not learn it in
# Introductory Programming 2. I am cooler :D
Date.print(today) # exposes the implicit object in the statement today.print()
```

```
2026/2/26
2026/2/26
```

4.7 A comparison to OOP in Python: Repeat last slide

```python
class Date:
  # blah blah

  def print(self):
    print(f"{self.year}/{self.month}/{self.date}")  # we don't have self. in C++
# Use this class in global scope
today = Date(2026,2,26) # when this lecture first took place
today.print()
Date.print(today)   # this exposes the implicit object in the statement today.print()
```

- What we have just learned in C++ looks different from Python you know. We don't have `self`: `self.year`, `self.month`, `self.day`.
- We shall learn an equivalence of `self` of Python in C++.
- Statement `Date.print(today)` exposes the implicit object in statement `today.print()`

4.8 A comparison to OOP in Python: But Python is not as serious as C++

```python
class Date:
  def __init__(self, year=2026, month=2, date=28): # we learn constructor in C++ soon
    self.year = year
    self.month = month
    self.date = date

  def print(self):
    print(f"{self.year}/{self.month}/{self.date}")  # we don't have self. in C++
# Use this class in global scope
today = Date(2026,2,26) # when this lecture first took place
my_name = "khiem"
Date.print(my_name) # SYNTAX Correct, but run time error. my_name.year does not exist
```

```
AttributeError: 'str' object has no attribute 'year'
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[23], line 12
     10 today = Date(2026,2,26) # when this lecture first took place
     11 my_name = "khiem"
---> 12 Date.print(my_name) # SYNTAX Correct, but run time error. my_name.year does not exist
Cell In[23], line 8, in Date.print(self)
      7 def print(self):
----> 8   print(f"{self.year}/{self.month}/{self.date}")
AttributeError: 'str' object has no attribute 'year'
```

4.9 A comparison to OOP in Python: But Python is not as serious as C++

```python
class Date:
  def __init__(self, year=2026, month=2, date=28): # we learn constructor in C++ soon
    self.year, self.month, self.date = year, month, date
  def print(self):
    print(f"{self.year}/{self.month}/{self.date}")  # we don't have self. in C++

class Foo:
  def __init__(self):
    self.year, self.month, self.date = "one", "two", 3
# Use this class in global scope
today = Date(2026,2,26) # when this lecture first took place
foo_object = Foo()

# SYNTAX correct, and Code run fine too. But it is just nonsense
Date.print(foo_object)
```

```
one/two/3
```

4.10 Member functions can be overloaded

Just like non-member functions, member functions can be overloaded.

4.11 Structs and member functions: A bit of history

- In C, structs only have data members, not member functions.
- While designing classes in C++, Bjarne Stroustrup considered whether structs (which were inherited from C) should be granted the ability to have member functions.
- Upon consideration, he determined that they should.
- In modern C++, it is fine for structs to have member functions.

Both structs and classes are called **class type**

5 Const class objects and const member functions

Again, we shall use `struct` for examples in this section.

Listing 4 Member functions can be overloaded. Filename=`overload_member_functions.cpp`

```cpp
#include <iostream>
#include <string_view>

struct Date
{
    int year {}; int month {}; int day {};  // Do not write like this! Save space for slide
    void print() {
        std::cout << year << '/' << month << '/' << day;
    }
    void print(std::string_view prefix) {
        std::cout << prefix << year << '/' << month << '/' << day;
    }
};

int main()
{
    Date today { 2020, 10, 14 };

    today.print(); // calls Date::print()
    std::cout << '\n';
    today.print("The date is: "); // calls Date::print(std::string_view)
    std::cout << '\n';

    return 0;
}
```

5.1 Const variables: A quick revisit

- We learn: All const variables must be initialized at time of creation

```cpp
const int x { 42 };
const double double z {};    // initialze to zero.
```

- Similarly, class type objects can be made const using `const` keyword

```cpp
struct Date
{
    int year {};
    int month {};
    int day {};
};
int main()
{
    // const class type object
    const Date today { 2020, 10, 14 };
    return 0;
}
```

5.2 Modifying the data members of const objects is disallowed

Any attempt to modify the data members of the object is disallowed.

Listing 5 Modifying the data members of const objects is disallowed. Filename=`modify_members_of_const_objects.cpp`

```cpp
struct Date
{   int year {}, month {}, day {};

    void increment_day() {
        ++day;
    }
};

int main()
{
    const Date today { 2020, 10, 14 }; // const
    today.day += 1;        // compile error: can't modify member of const object
    today.increment_day();  // compile error: can't call member function that modifies member of c

    return 0;
```

Show the code with uncommented code. IntelliSense picked up the error before compilation.

5.3 Const objects may not call non-const member functions

You may be surprised that the following code causes compilation error.

Listing 6 const object calling the non-const member functions will create compilation error.

```cpp
#include <iostream>

struct Date
{
    int year {}, month {}, day {};
    void print()     {
        std::cout << year << '/' << month << '/' << day;
    }
};

int main()
{
    const Date today { 2020, 10, 14 }; // const
    // today.print();  // compile error: can't call non-const member function
    return 0;
}
```

5.4 Const member functions

To address the above issue, we make `print()` a const member function.

A **const member function** is a member function that guarantees it will not modify the object or call any non-const member functions (as they may modify the object)

Listing 7 const member function can be called by const objects. Filename=`const_member_functions.cpp`

```cpp
#include <iostream>

struct Date
{
    int year {}, month {}, day {};
    void print() const // now a const member function
    {
        std::cout << year << '/' << month << '/' << day;
        double another_year {42};
        another_year += 1;
    }
};

int main()
{
    const Date today { 2020, 10, 14 }; // const
    today.print();  // ok: const object can call const member function

    return 0;
}
```

5.5 Const member functions

Not a surprise: A const member function that attempts to change a data member or call a non-const member function will cause a compiler error to occur.

5.6 Const member functions can be called on non-const objects

Not a surprise (again): Const member functions may also be called on non-const objects.

- We promise the compiler *const member functions won't change the data members.*
- Indeed, non-const objects calling const member functions cannot break such promise.

5.7 Const member functions can be called on non-const objects

5.8 Const member functions can be called on non-const objects

A member function that does not (and will not ever) modify the state of the object should be made const, so that it can be called on both const and non-const objects.

Listing 8 A const member function cannot change data members. Filename=`const_member_functions_change_data_members.cpp`

```cpp
struct Date
{
    int year {}, month {}, day {};

    void increment_day(int val) const // made const
    {
        // day += val; // compile error: const function can't modify member
    }
};

int main()
{
    const Date today { 2020, 10, 14 }; // const

    today.increment_day(4);

    return 0;
}
```

5.9 Wait a minute: What a mess

- How am I supposed to remember all the const-ness thingys.
- I am not even Tony Stark or Spider Man.

So, I asked Gemini:

There are too many things to remember about making function const and making reference const in writing C++ code to deal with const objects. Can you please give me one or two single golden rule so that I remember what I should do?

Gemini answers me (I shortened it a bit):

If you are working with **structs** or **classes**, here are the two golden rules that will cover 95% of your needs.

Rule 1 The "Read-Only" Rule (For Functions)

If a function does not change the object's data, mark it `const` at the end.

Think of it as a promise to the compiler.

Rule 2 The "Efficient Pass" Rule (For Parameters)

Always pass objects by `const &` (const reference) unless they are tiny or we need to modify its value.

Let us combine both rules into one example

5.10 Const objects via pass by const reference

Recap:

- Passing a class type argument by value causes a copy of the class to be made – which is slow.
- **Passing by reference avoids making a copy.**

Note

Listing 9 A non-const object can call const member functions because it cannot break the promise of constant-ness. Filename=`nonconst_objects_call_const_mem_functions.cpp`

```cpp
#include <iostream>

struct Date
{
    int year {}, month {}, day {};
    void print() const {// const
        std::cout << year << '/' << month << '/' << day;
    }

};
int main()
{
    Date today { 2020, 10, 14 }; // non-const
    today.print();  // ok: can call const member function on non-const object

    return 0;
}
```

- Note that modifying the argument's value is not the key point. It is a side effect.
- If we add `const` qualifier, we cannot modify the arguments' value.
- Of course, *we wish to modify the argument's value in reasonable cases.*

Another Note

- Instantiating const local variables is one way to create const objects
- Another common way to get a const object is *by passing an object to a function by const reference. But, this only holds from the perspective of that function.*

5.11 Const objects via pass by const reference

Question What's wrong with the following code?

Compile the file and show the code in VSCode.

5.12 Const objects via pass by const reference

Yup, the fix is simple: make `print()` const

Listing 10 Passing an object by const reference make the object const in the perspective of the function. Filename=`const_objects_via_pass_by_const_ref_01.cpp`

```cpp
#include <iostream>

struct Date
{
    int year {}, month {}, day {};
    void print() {  // non-const
        std::cout << year << '/' << month << '/' << day;
    }
};

void do_something(const Date& date) {
    date.print();
}

int main()
{
    Date today { 2020, 10, 14 }; // non-const
    today.print();

    do_something(today);

    return 0;
}
```

Listing 11 Fix the above code listing by making `print()` const. Filename=`const_objects_via_pass_by_const_ref_01.cpp`

```cpp
#include <iostream>

struct Date
{
    int year {}, month {}, day {};
    void print() const {  // non-const
        std::cout << year << '/' << month << '/' << day;
    }
};

void do_something(const Date& date)
{
    date.print();
}

int main()
{
    Date today { 2020, 10, 14 }; // non-const
    today.print();

    do_something(today);

    return 0;
}
```